LEDGER: Execution Provenance Auditing for Data-Agent Answers

Anonymous Authors

Abstract

Data agents answer questions by selecting sources, writing SQL or Python programs, executing them, and reporting computed values. Auditing such answers requires binding the stated answer to the final program, the evidence declared by the agent, and the data actually read during execution. We present LEDGER, a deterministic post-execution audit layer. Given a completed episode, it canonicalizes declared sources, reconciles observed reads, replays the final program inside a citation-restricted evidence view, and returns a replay certificate, a localized failure, or an unknown status. When the runtime is unavailable, bounded closure over trace-local operands supplies a derivation certificate. A task-specific semantic validator independently checks whether the program implements the requested operation. We formalize the trace contract, trust assumptions, and the relative guarantee provided by replay. Across ConvFinQA and MultiHiertt, trace-local closure improves balanced accuracy by 9.3–18.0 points over single-operation checks, while generic anomaly detectors identify only 13.7–19.8% of unsupported constructions. A controlled WikiSQL study shows that same-output program edits evade output equality, whereas normalized structure detects every controlled edit without rejecting paired originals. On human-labelled KramaBench episodes, replay-based delivery raises support precision by 24.2 points; on scored live episodes, it improves delivered utility at 64.2% coverage. The audit is deterministic, makes zero model calls, and adds sub-millisecond bookkeeping on saved traces. Execution-provenance auditing therefore complements semantic validation when deciding which computed answers to deliver or escalate.

Keywords: data agents, execution provenance, auditability, selective prediction, semantic validation, data lakes

1. Introduction

Data agents convert natural-language questions into executable data pipelines. They search a data lake, inspect schemas, write SQL or Python, repair execution errors, and submit a value with supporting sources [1, 2, 3]. This workflow changes the object of verification. A reported value may occur in no source file because it is produced by filtering, joining, and aggregation. Evidence presence alone therefore does not establish how the answer was obtained.

A KramaBench episode illustrates the gap. The question requests the average number of identity-theft reports per 100,000 population across metropolitan areas. The agent cites 17 files and states 10,724.14. Its final program executes, yet replay under the cited-file view produces no value comparable to the stated answer. A citation checker sees relevant files; a post-execution audit observes that the recorded computation does not reproduce the claim.

Replay also leaves a second question open. A program may average the wrong years, select the wrong subset, or compute a sum when the question requests a mean. The trace can reproduce the stated value while the program implements the wrong operation. We therefore separate two properties. *Execution-provenance consistency* asks whether the final program reproduces the stated answer from authorized evidence. *Semantic correctness* asks whether that program implements the task specification. The first property admits deterministic execution witnesses. The second depends on a task-specific specification, such as reference SQL, test suites, metamorphic constraints, or human review.

Existing methods cover adjacent objects. Citation metrics evaluate textual support [4, 5]. Database provenance represents how query outputs depend on inputs [6, 7, 8]; script-provenance systems capture runtime dependencies in controlled environments [9, 10]. Agent benchmarks measure utility and risk under tool use [11, 12], and data-agent benchmarks increasingly expose process failures [1, 13]. A completed data-agent episode still needs a compact audit that binds its answer to its final execution and self-declared evidence boundary.

We develop LEDGER for this purpose. It runs after the host agent has finalized an episode. Citation-restricted replay is the primary witness; bounded operand closure is a fallback for archived traces without a runnable environment. The audit returns a replay or closure certificate, a localized failure, or an unknown status. An independent semantic status then supports delivery or escalation without changing the audit result.

The paper makes four contributions.

1. We formulate post-execution auditing for computed data-agent answers with an explicit trace contract, trust model, and independent provenance and semantic statuses.
2. We develop a deterministic audit procedure that resolves declared evidence, reconciles host reads, performs citation-restricted replay, and returns reproducible witnesses or localized failures.
3. We derive the relative evidence-boundary guarantee of replay, establish completeness of the bounded fallback, and formalize why provenance status does not imply program

intent.

4. We compare ordinary rules, arithmetic checks, anomaly detectors, and a local model judge across five testbed families, then evaluate semantic validation and selective delivery on controlled programs and live episodes.

2. Audit Problem and Trust Model

2.1. Completed episode and trace contract

A completed data-agent episode is

$$E = (q, \mathcal{D}, \tau, \hat{a}),$$

where q is the question, $\mathcal{D}$ is the accessible data environment, $\hat{a}$ is the stated answer, and τ is the final host trace. We write

$$\tau = (P^*, F_{\text{decl}}, F_{\text{host}}, \Omega, s_{\text{host}}, v_{\text{host}}).$$

Here P^* is the final program snapshot, F_{decl} is the evidence declared by the agent, F_{host} contains observed data-source reads in the instrumented namespace, Ω contains values reached by the final computation, and $(s_{\text{host}}, v_{\text{host}})$ records host execution status and comparable output when available. Program versions, tool calls, exceptions, and timestamps remain in the full record but are omitted from the tuple.

A canonical resolver maps declarations into an evidence view

$$F_{\text{res}} = R(F_{\text{decl}}) = \{\text{canon}(f) : f \in F_{\text{decl}},\ f \text{ resolves}\}.$$

During audit, restricted replay produces

$$\rho = (F_{\text{replay}}, s_{\text{replay}}, v_{\text{replay}}),$$

where F_{replay} records reads observed inside the replay sandbox. Keeping host reads and replay reads separate distinguishes an undeclared dependency in the original episode from a boundary violation during verification.

The audit output is

$$\mathsf{Audit}(E) \in \{\mathsf{cert}_{\text{replay}}, \mathsf{cert}_{\text{closure}}, \mathsf{failure}(c), \mathsf{unknown}\}.$$

A failure records an observed violation and its witness; unknown records insufficient evidence or an unavailable runtime. This distinction prevents missing evidence from being interpreted as a detected error.

2.2. Independent provenance and semantic statuses

The provenance status is produced by Ledger from τ, R, a restricted runtime, and a frozen comparator. Semantic validation receives a task-specific specification V_q and returns

$$S(E; V_q) \in \{\text{correct}, \text{incorrect}, \text{unknown}\}.$$

Depending on the task, V_q may contain reference SQL, a normalized program specification, database variants, executable constraints, metamorphic relations, or a human judgement. The provenance and semantic statuses are stored as independent fields. Neither is used as an input to the other.

2.3. Threat and trust assumptions

The host agent, generated program, stated answer, and declared citations are fallible. The trusted computing base consists of the trace logger, canonical resolver, restricted replay sandbox, and type-aware comparator. The analysis assumes that the logger captures the final program snapshot and all data-source reads in the instrumented namespace, the resolver maps identifiers consistently, and the sandbox enforces its mounted evidence view. These assumptions make the guarantee relative to recorded execution evidence.

Tampering with the logger, sandbox escape, nondeterministic external services, and dependencies outside the instrumented namespace are outside the present guarantee. Such events should yield unknown or a platform-level security failure rather than an execution-provenance certificate. This scope aligns the paper with post-execution auditing: it provides action recoverability and evidence integrity for computed answers, while lifecycle-wide accountability requires additional controls [22].

3. What a Post-Execution Audit Must Establish

The motivating episode exposes three distinct decisions. The audit must identify which evidence the host computation was authorized to use, determine whether the final program reproduces the stated answer under that evidence view, and preserve the boundary between reproducibility and task intent. These decisions motivate the Ledger design in Section 4.

3.1. Which evidence is authorized?

An evidence view is complete when every declaration resolves. A completed host episode is evidence-authorized when

$$F_{\text{host}} \subseteq F_{\text{res}}.$$

An unresolved declaration and an undeclared host read are different failures: the first leaves a claimed source unavailable, whereas the second shows that the recorded computation depended on data outside the claimed boundary. Canonicalization occurs before either comparison so aliases and path variants do not create artificial mismatches.

3.2. Does the recorded execution reproduce the answer?

Let $H[F_{\text{res}}]$ denote a replay environment that exposes only the resolved evidence view, and let $\text{cmp}(\cdot, \cdot; \varepsilon)$ be a frozen type-aware comparator. A replay certificate is issued exactly when

$$\begin{aligned} \text{Cert}_{\text{replay}}(E) = 1 \iff\ & F_{\text{res}} = R(F_{\text{decl}}) \text{ is complete}, \\ & F_{\text{host}} \subseteq F_{\text{res}}, \quad s_{\text{host}} = \text{success}, \\ & \rho = \text{Replay}(P^*, H[F_{\text{res}}]), \\ & s_{\text{replay}} = \text{success}, \quad F_{\text{replay}} \subseteq F_{\text{res}}, \\ & \text{cmp}(v_{\text{replay}}, \hat{a}; \varepsilon) = 1. \end{aligned} \tag{1}$$

The witness stores the program snapshot, resolved evidence identifiers, host and replay reads, replay output, comparator, and tolerance.

Theorem 1 (Relative soundness of replay)**.** *Under the trust assumptions of Section 2.3, a replay certificate establishes that the stated answer is reproduced by P^* using only resolved declared evidence, and that no observed host or replay data dependency lies outside that evidence view. Any unresolved declaration, undeclared host read, replay boundary violation, execution failure, missing comparable output, or answer mismatch prevents the certificate.*

Proof sketch. Equation 1 requires evidence resolution, read authorization, successful restricted execution, and output agreement. The trusted logger and sandbox make the two read-set inclusions observable and enforceable. Violating any conjunct follows a failure branch in Algorithm 1; the full argument appears in Appendix A.

3.3. What remains outside provenance auditing?

A replay witness describes the program that ran. It does not compare that program with the operation requested by q. The same limitation applies to arithmetic derivability when a different operation over the same operands reaches the same value.

Proposition 1 (Semantic non-implication)**.** *There exist episodes for which* $\mathrm{Cert}_{\mathrm{replay}}(E) = 1$ *or* $\hat{a} \in \mathrm{Cl}_d(\Omega)$ *while* $S(E; V_q) =$ incorrect*. Consequently, execution-provenance status alone does not imply semantic correctness.*

A sum and a mean can coincide on a particular database, and a wrong filter can leave the current output unchanged. This property motivates the independent semantic branch in Figure 1 and the same-output SQL evaluation in Section 5.4.

Algorithm 1 LEDGER episode audit

Require: episode E, resolver R, sandbox H, comparator cmp, operations O, depth d

$F_{\mathrm{res}} \leftarrow R(F_{\mathrm{decl}})$
if some declaration is unresolved **then**
  return failure(unresolved source)
end if
if $F_{\mathrm{host}} \not\subseteq F_{\mathrm{res}}$ **then**
  return failure(undeclared host read)
end if
if $s_{\mathrm{host}} \neq$ success and an answer was stated **then**
  return failure(host execution)
end if
if P^* is runnable in H **then**
  $(F_{\mathrm{replay}}, s_{\mathrm{replay}}, v_{\mathrm{replay}}) \leftarrow \mathrm{Replay}(P^*, H[F_{\mathrm{res}}])$
  if $F_{\mathrm{replay}} \not\subseteq F_{\mathrm{res}}$ **then**
    return failure(replay boundary)
  else if $s_{\mathrm{replay}} \neq$ success **then**
    return failure(replay execution)
  else if v_{replay} is not comparable **then**
    return failure(missing output)
  else if $\mathrm{cmp}(v_{\mathrm{replay}}, \hat{a}; \varepsilon) = 1$ **then**
    return $\mathsf{cert}_{\mathrm{replay}}$ and replay witness
  else
    return failure(output mismatch)
  end if
end if
if reached operands are complete **then**
  return $\textsc{BoundedClosure}(\Omega, O, d, \hat{a})$
end if
return unknown

4. The LEDGER Audit Layer

4.1. Overview and trace capture

The host wrapper records program revisions, the final program snapshot, stdout, exceptions, declared citations, runtime data reads, accessed columns, and scalar operands reaching the final expression. Paths and dataset identifiers are canonicalized before comparison. Declared sources, host reads, and replay reads remain separate fields throughout the audit record.

For dataframe programs, operand attribution follows accessed columns and scalar values that reach the final expression. For SQL, the record stores relations, projected attributes, predicates, aggregates, and returned scalars. Each outcome carries a concise evidence string and the machine-readable artifact needed to reproduce it.

4.2. Episode audit and failure precedence

Algorithm 1 implements the complete audit. Evidence-boundary failures take precedence over replay failures, which take precedence over output mismatch. When several violations are observed, the record preserves the full set and exposes the earliest class as the primary outcome. Closure is consulted only when no explicit failure has been observed and the final runtime is unavailable.

4.3. Citation-restricted replay

The replay sandbox mounts only F_{res}, fixes deterministic library settings when available, and records every data read. The comparator normalizes strings, booleans, ordered or unordered lists, and numeric values under relative tolerance ε. A successful certificate therefore binds a concrete program snapshot, a canonical evidence view, and a comparable output. Failure records retain the denied read, exception, missing output, or mismatching value that caused the decision.

4.4. Operand-constrained fallback

Some archived episodes expose operands and a stated value while the original runtime is unavailable. Let

$$O = \{\mathrm{sum}, \mathrm{diff}, \mathrm{ratio}, \mathrm{pctchg}, \mathrm{prod}\}.$$

Starting from the trace-local operand multiset Ω, the depth-d closure $\mathrm{Cl}_d(\Omega)$ recursively applies valid operations in O with normalization and duplicate removal.

Definition 1 (Closure certificate)**.** *Episode E receives a depth-d closure certificate when $\hat{a} \in \mathrm{Cl}_d(\Omega)$ within tolerance ε. The witness stores a minimal expression and its operand subset.*

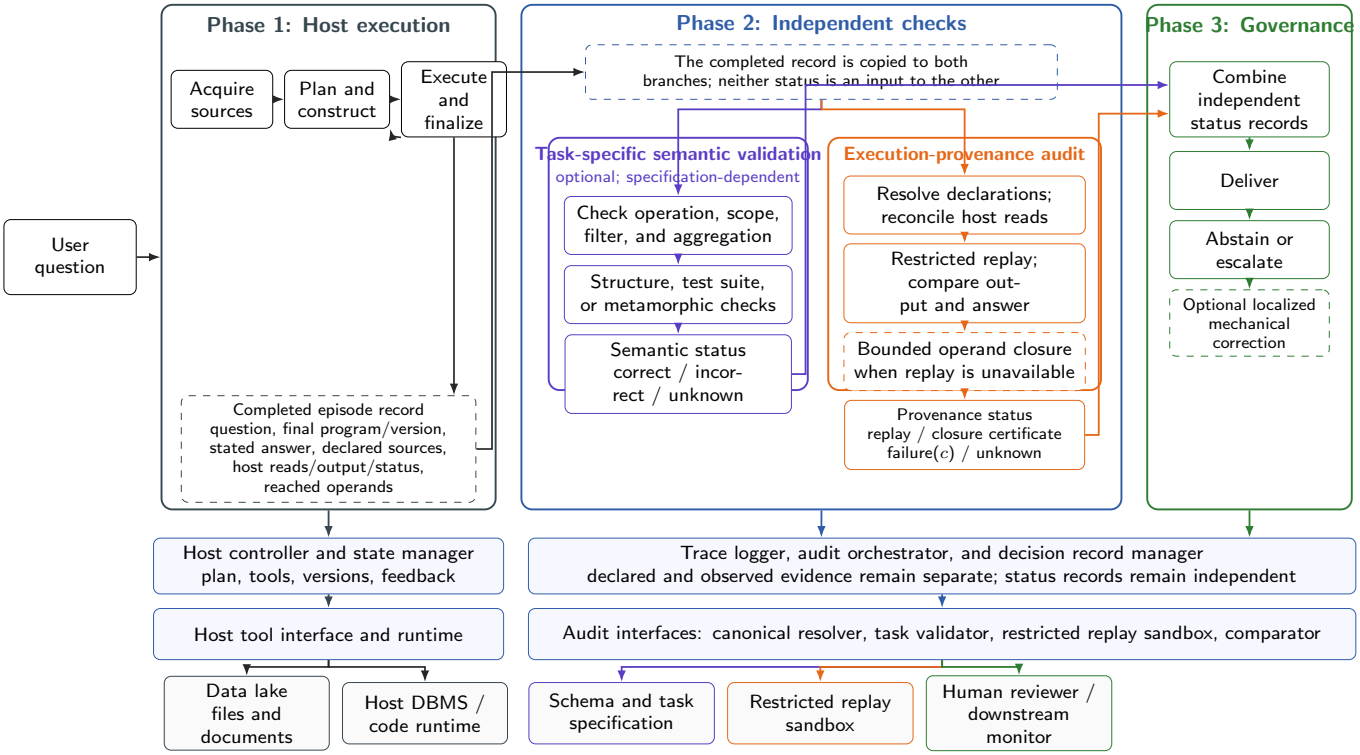


Figure 1: LEDGER consumes a completed episode after host execution. Semantic validation and execution-provenance auditing run independently; dashed boxes denote the bounded fallback and an optional correction path that is not evaluated here.

Algorithm 2 Bounded operand closure

function BoundedClosure($\Omega, O, d, \hat{a}$)
  $S_0 \leftarrow$ normalize(Ω)
  for $j = 1, \ldots, d$ **do**
    $S_j \leftarrow S_{j-1}$
    for all $x, y \in S_{j-1}$ and $o \in O$ **do**
      add valid normalized $o(x, y)$ with an expression witness
    end for
  end for
  if $\hat{a} \in S_d$ within ε **then**
    return $\text{cert}_{\text{closure}}$ and a minimal witness
  end if
  return unknown
end function

Proposition 2 (Closure completeness)**.** *If $\hat{a}$ is generated from reached operands by operations in O, expression depth at most d, and the audit's normalization and tolerance, then $\hat{a} \in \text{Cl}_d(\Omega)$.*

Proof sketch. Leaves occur in S_0. Induction on expression depth places each valid internal result in the next closure level. The root therefore appears by depth d; The full proof appears in Appendix A.

The operating point is calibrated on 60 disjoint ConvFinQA examples and frozen at $d = 2$ with relative tolerance 10^{-4}. Context-wide numbers, held-out examples, and human labels do not change this setting.

4.5. Semantic interface and governance decision

The audit record exposes q, P^*, schema references, and replay output to the optional validator V_q. SQL tasks permit normalized-structure comparison and execution on generated database variants [14]; other deployments may supply executable constraints, metamorphic relations, a model-based judge, or human review. The resulting semantic status is appended to the audit record without modifying its provenance outcome.

A conservative decision policy delivers a provenance-certified answer when the semantic status is correct or unavailable under the deployment policy, and escalates semantically rejected or provenance-incomplete episodes. A uniquely localized mechanical mismatch can support an optional correction workflow; the present evaluation measures delivery and escalation and does not evaluate automated repair.

Remark 1. *The provenance witness remains valid without a semantic validator. Semantic status is a separate, deployment-specific property.*

4.6. Complexity and deterministic contract

Resolution and read reconciliation cost $O(|F_{\text{decl}}| + |F_{\text{host}}|)$. Replay adds one execution of the final program. If N_j is the number of closure candidates before duplicate removal, $m = |\Omega|$, and $r = |O|$, then $N_d \leq m + r \sum_{j=1}^{d} N_{j-1}^2$. Restricting S_0 to reached operands reduces every recursive expansion. For a fixed trace, resolver, runtime, comparator, normalization, and depth, the provenance audit is deterministic and invokes no language model.

Table 1: Interface between independent statuses and downstream action.

Provenance status	Semantic status	Default action
certificate	correct	deliver
certificate	unknown	deliver or review by policy
certificate	incorrect	abstain / escalate
failure or unknown	any	abstain / escalate
localized mismatch	correct/unknown	optional correction

5. Evaluation

5.1. Questions and evidence layers

We ask four questions.

RQ1 How does operand-constrained auditing compare with ordinary numeric, trace-rule, anomaly, and model-based checks?

RQ2 Does the frozen audit transfer across datasets, and how does closure depth expose its recall–collision trade-off?

RQ3 Which semantic changes remain invisible to output replay, and what complementary checks recover them?

RQ4 Does replay-certificate delivery improve the quality of answers sent to users or downstream systems?

Table 2 summarizes five testbed families. Program-labelled corpora provide construction-grounded supported and unsupported pairs. WikiSQL provides controlled program perturbations. KramaBench and DABstep provide completed live traces. Human annotation covers 40 KramaBench episodes; one unclear case is omitted from binary analysis.

Ordinary baselines. Numeric baselines include exact operand matching, one-operation derivability, and closure over all numbers in the cited context. Trace rules check execution success, citation resolution, declared-read coverage, and unrestricted replay. Anomaly baselines use robust MAD distance, Isolation Forest [15], Local Outlier Factor [16], and one-class SVM [17]. Their thresholds use only the 60-example calibration seed and retain at least 90% calibration support recall. A supervised logistic model provides an upper baseline. No anomaly feature contains the audit verdict or benchmark answer.

Evidence-aware judge. A local Qwen3.6-35B judge receives the question, declared evidence metadata, reads, program summary, execution status, recomputed output, and stated answer. It receives neither benchmark gold nor LEDGER labels. Three temperature-zero repeats quantify response stability. Transferred human records omit most raw rows and full program bodies, so that subset remains a limited-evidence diagnostic.

5.2. RQ1: Trace-local operands separate derivations from numeric coincidence

Trace-local operands preserve multi-step derivations while suppressing unrelated numbers. ConvFinQA supplies 386 evaluable supported derivations and aligned cross-example unsupported answers; MultiHiertt supplies 6,301 aligned pairs. The frozen checker uses reached operands, five operations, depth 2, and relative tolerance 10^{-4}. Compared with a single-operation check, LEDGER raises balanced accuracy by 9.3 points on ConvFinQA and 18.0 points on MultiHiertt. Closure over all cited-context numbers recovers more supported cases than one-step checking but admits many accidental collisions. Exact counts appear in Table 3 and Figure 2(a,b).

Generic anomaly scores do not establish a computational witness. Isolation Forest, Local Outlier Factor, and one-class SVM retain most supported cases but detect only 13.7–19.8% of unsupported constructions. Their trace features identify unusual executions; they do not show that an answer has a derivation from the reached values. Restricting closure to program-reached operands preserves the relevant search space and removes unrelated numbers before recursive expansion. The same operating point transfers from conversational finance programs to longer hierarchical table-and-text derivations without retuning.

5.3. RQ2: Depth recovers derivations and expands the semantic collision surface

The frozen depth captures short programs; longer programs account for most remaining misses. We stratify MultiHiertt by gold program depth and vary closure depth on the disjoint ConvFinQA calibration seed. Recall is highest for programs within the frozen bound, and 72.3% of MultiHiertt misses exceed depth 2. Raising the calibration depth from 1 to 3 recovers every supported seed derivation, while cross-example rejection drops sharply (Figure 3). The depth-2 setting therefore remains a bounded operating point rather than an attempt to enumerate arbitrary programs.

Depth enlarges the semantic collision surface. We replace each ConvFinQA answer with a different operation over the same operands. At the frozen vocabulary and depth, none of 347 in-scope wrong-operation substitutions is rejected. Deeper closure accepts still more such variants because the altered answers remain derivable from the same values. Cross-example swaps remain much easier to detect because they are generally unrelated to the reached operands. This contrast instantiates Proposition 1: closure records arithmetic derivability, while operation identity belongs to the semantic validator.

5.4. RQ3: Structural validation recovers same-output program changes

Output equality under-detects controlled semantic edits. From 500 executable WikiSQL development examples, we construct 2,715 perturbations that change aggregation, projection, predicate column or value, and predicate presence. The study is a controlled semantic pilot rather than an estimate of natural agent-error prevalence. Output equality misses 466 edits because the modified and reference programs happen to return the same result on the original database.

Program structure complements replay on the same-output subset. Normalized SQL structure detects every controlled edit and accepts every paired unmodified query. On a 100-case same-output sample, operator/clause signatures detect 67 edits and differential execution detects 27, while replay detects none (Figure 2c). A single denotation can remain unchanged after an operation or predicate edit; structural comparison observes

Table 2: Evaluation layers. The five families isolate arithmetic derivability, SQL semantics, live execution provenance, and human-facing delivery.

Layer	Dataset	Total	Evaluable	Primary use
Program-labelled	ConvFinQA remainder	388	386	calibration-free arithmetic audit and baselines
Program-labelled	MultiHiertt	7,830	6,301	transfer and derivation-depth stress
Controlled SQL	WikiSQL	500 queries	2,715 perturbations	same-output semantic changes
Live traces	KramaBench	166	166	replay, failures, selective delivery
Live traces	DABstep	60	60	selection-free transfer
Human labels	KramaBench sample	40	39	support, semantic boundary, escalation

Table 3: ConvFinQA ordinary baselines on the held-out remainder.

Method	Support recall	Unsupported detection
Single operation	276/386 (0.715)	383/386 (0.992)
All cited numbers	332/386 (0.860)	240/386 (0.622)
Isolation Forest	351/388 (0.905)	74/388 (0.191)
Supervised upper	349/388 (0.899)	230/388 (0.593)
LEDGER	**360/386 (0.933)**	**371/386 (0.961)**

the changed program directly, whereas test-suite execution succeeds only when generated database variants distinguish the two programs. SQL deployments can therefore instantiate V_q with structure and test-suite methods [14]; model-based metrics such as FLEX remain a complementary option when reference programs are unavailable [18].

5.5. RQ4: Replay certificates support selective delivery

Replay status provides a human-facing delivery signal. The live corpus contains 166 KramaBench episodes and 60 DABstep episodes. The human subset contains 39 clear labels. Restricting delivery to replay-certified cases retains every human-supported episode, escalates most unsupported episodes, and raises support precision by 24.2 points (Figure 4a,b). The replay-certified unsupported cases compute a reproducible value through a program that addresses the wrong quantity, scope, or operation, which again separates provenance from semantics.

The same signal improves utility on scored live episodes. Among the 148 KramaBench episodes with finite official scores, replay delivery operates at 64.2% coverage and raises mean delivered score from 0.307 to 0.429; uniform random selection at the same size remains at 0.307. DABstep shows the same boundary because several benchmark-incorrect answers remain replayable. The policy improves which completed episodes are delivered; it does not change source selection, program generation, or host-agent execution.

5.6. Runtime, determinism, and model baseline

Deterministic bookkeeping is small relative to host execution. Across 500 saved records, the audit adds sub-millisecond median bookkeeping and makes zero model calls. Five independent repeats over 100 records produce identical provenance outcomes. These measurements cover record parsing, comparison, and closure over saved traces; they exclude host-agent execution and external data loading.

An evidence-aware 35B judge provides a model-based diagnostic but shows label disagreement, structured-output failures, and occasional repeat flips. The complete counts appear in Appendix Appendix B. Judge outputs remain separate from the deterministic provenance status.

6. Related Work

Data-agent evaluation.. KramaBench evaluates data-to-insight pipelines over heterogeneous lake files [1]; DABstep targets multi-step tool-based analysis [2]. LakeQA scales discovery and reasoning to a million-scale lake [3]. SANA decomposes failures across search, planning, analysis, and policy [13]. These benchmarks motivate process-level evidence in addition to final-answer accuracy.

Execution provenance and auditability.. Database provenance formalizes how outputs depend on inputs [6, 7, 8]. noWorkflow and Vamsa capture dependencies in data-science scripts [9, 10]. Agent-trace work organizes retrieved evidence, tool outputs, memory, actions, and final claims under process-level accountability [19]. Auditable Agents distinguishes accountability, auditability, and auditing and identifies action recoverability and evidence integrity as core dimensions [22]. LEDGER operationalizes these dimensions for completed computed answers and their declared evidence view.

Agent governance, security, and trace evaluation.. Contextual security frameworks separate task alignment, action alignment, source authorization, and data isolation [23]; intent-to-execution integrity further separates tool, instruction, judgment, and data-flow properties [24]. The provenance status in LEDGER covers an observed source and execution boundary, while its task-specific semantic interface addresses a separate alignment relation. TrustAgent intervenes before, during, and after planning through an agent constitution [25]; LEDGER instead audits a completed episode. ToolEmu, AgentDojo, and R-Judge establish tool trajectories as standard evaluation objects [11, 12, 20].

Citation, semantic, and selective evaluation.. ALCE and FActScore evaluate textual citation support and factual claims [4, 5]. Text-to-SQL test suites approximate semantic equivalence through execution on multiple database variants [14]; FLEX uses an LLM-based expert metric to reduce execution-metric errors [18]. Selective QA trades coverage for reliability using confidence [21]. LEDGER derives its delivery signal from a replay witness and retains task-specific semantic evaluation as an independent control.

7. Limitations

The guarantee is relative to the trusted logger, resolver, sandbox, and comparator. Tamper-evident logging, sandbox escape, nondeterministic network services, and unobserved dependencies remain outside the current threat model. Platform-level controls are needed before the same certificate semantics can be applied in adversarial environments.

The arithmetic fallback supports a frozen five-operation vocabulary and depth 2. It omits string transformations, joins, and long programs when replay is unavailable. Increasing depth improves derivation coverage and rapidly increases collisions. Current replay adapters cover the Python/pandas and SQL traces retained by the testbed families; additional formats require runtime-specific adapters.

Human support is available for 39 clear KramaBench cases. A larger independently annotated corpus would strengthen estimates of deployment prevalence. The WikiSQL study uses controlled perturbations and access to reference programs. It demonstrates the complementarity of replay and structural validation; it does not establish universal natural-language-to-program equivalence. Automated mechanical correction is shown only as a downstream interface and is not evaluated in this paper.

8. Conclusion

Computed data-agent answers require evidence that spans declared sources, observed reads, final programs, execution outcomes, and stated values. Ledger converts a completed episode into a deterministic execution-provenance record through source resolution, read reconciliation, citation-restricted replay, and bounded operand closure. Ordinary rules and anomaly detectors do not match its joint derivation recall and unsupported-answer detection. Live traces show that replay certificates support selective delivery while preserving an explicit unknown path. Controlled SQL perturbations establish the complementary role of task-specific semantic validation. Together, the two independent statuses provide a reproducible basis for delivery and escalation decisions.

Appendix A. Proofs

Appendix A.1. Theorem 1

A replay certificate is returned only after every declaration resolves, both observed read sets lie within the resolved evidence view, host and replay execution succeed, and the frozen comparator matches replay output to the stated answer. Under the trust assumptions, these observations correspond to the recorded data dependencies and the sandbox-enforced execution. Each listed violation negates at least one required conjunct in Equation 1 and follows a failure branch in Algorithm 1; hence no replay certificate is returned.

Appendix A.2. Proposition 2

At depth zero, every leaf is a reached operand and belongs to S_0. Assume every valid expression of depth at most $j-1$ is represented in S_{j-1}. An expression of depth j has the form $o(e_1, e_2)$ with $o \in O$ and child depth at most $j-1$. The induction hypothesis places both child values in S_{j-1}; Algorithm 2 adds their valid normalized result to S_j. The root therefore appears by depth d.

Appendix A.3. Proposition 1

Replay compares the submitted program's output with the stated answer under the declared evidence view. Closure tests whether the answer has an admitted derivation from reached operands. Neither predicate compares the submitted program with V_q. Choose two programs with different task semantics but identical output on the current database, or two different admitted arithmetic expressions with the same result. The provenance predicate holds while the semantic validator rejects the altered program.

Appendix B. Additional Result Tables

Table B.4: Canonical live audit distributions.

Dataset	Total	Replay	Failure	Unknown
KramaBench	166	107	3	56
DABstep	60	43	0	17

Table B.5: Evidence-aware local judge. Agreement uses construction or human labels; the human row has limited archived evidence.

Dataset	Agreement	Flip items	Parse failures
ConvFinQA constructed	170/200	4/200	
KramaBench human	25/39	1/39	29/870
WikiSQL perturbations	44/50	3/50	

Appendix C. Human Annotation and Reproducibility

The 40-row sample spans archeology, environment, legal, and wildfire domains. Annotators receive the question, cited files, final program or available program summary, stated answer, replay output, and evidence notes. The support label asks whether the stated answer follows from the cited data through the recorded computation and whether that computation addresses the requested quantity. The benchmark answer and machine label are hidden. Labels are yes, no, and unclear; one unclear case is excluded from binary analysis.

Machine outputs are frozen before annotation. Human columns remain in a separate file and are never used to rewrite audit labels. The reproducibility package contains the operating point, per-example arithmetic records, live trace records, semantic perturbations, baseline scripts, figure-generation code, and SHA-256 manifests. Every rate in the paper is recoverable from machine-readable counts.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used OpenAI ChatGPT to assist with manuscript restructuring, language editing, figure preparation, and LaTeX preparation. The authors reviewed and edited the resulting content and take full responsibility for the publication.

References

[1] E. Lai, G. Vitagliano, Z. Zhang, S. Sudhir, O. Chabra, A. Zeng, A. A. Zabreyko, C. Li, F. Kossmann, J. Ding, J. Chen, M. Markakis, M. Russo, W. Wang, Z. Wu, M. J. Cafarella, L. Cao, S. Madden, T. Kraska, KramaBench: A benchmark for AI systems on data-to-insight pipelines over data lakes, in: International Conference on Learning Representations, 2026, arXiv:2506.06541.

[2] A. Egg, M. Iglesias Goyanes, F. Kingma, A. Mora, L. von Werra, T. Wolf, DABstep: Data agent benchmark for multi-step reasoning, arXiv:2506.23719 (2025).

[3] H. Wang, J. Liu, Y. Liu, A. S. Wijaya, T. Zhou, E. Wu, Y. Chen, W. You, R. Vir, D. Pinto Veizaga, G. Fan, Y. Zhang, J. Freire, E. Wu, LakeQA: An exploratory QA benchmark over a million-scale data lake, in: Proceedings of the 43rd International Conference on Machine Learning, Vol. 306 of Proceedings of Machine Learning Research, 2026, arXiv:2606.10460.

[4] T. Gao, H. Yen, J. Yu, D. Chen, Enabling large language models to generate text with citations, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 6465–6488. doi:10.18653/v1/2023.emnlp-main.398.

[5] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 12076–12100. doi:10.18653/v1/2023.emnlp-main.741.

[6] T. J. Green, G. Karvounarakis, V. Tannen, Provenance semirings, in: Proceedings of the ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, 2007, pp. 31–40. doi:10.1145/1265530.1265535.

[7] J. Cheney, L. Chiticariu, W.-C. Tan, Provenance in databases: Why, how, and where, Foundations and Trends in Databases 1 (4) (2009) 379–474. doi:10.1561/1900000006.

[8] Y. Amsterdamer, D. Deutch, V. Tannen, Provenance for aggregate queries, in: Proceedings of the ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, 2011, pp. 153–164. doi:10.1145/1989284.1989302.

[9] L. Murta, V. Braganholo, F. Chirigati, D. Koop, J. Freire, noWorkflow: Capturing and analyzing provenance of scripts, in: Proceedings of the International Provenance and Annotation Workshop, 2014, pp. 71–83. doi:10.1007/978-3-319-16462-5_6.

[10] M. H. Namaki, A. Floratou, F. Psallidas, S. Krishnan, A. Agrawal, Y. Wu, Y. Zhu, M. Weimer, Vamsa: Automated provenance tracking in data science scripts, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2020, pp. 1542–1551. doi:10.1145/3394486.3403205.

[11] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, T. Hashimoto, Identifying the risks of LM agents with an LM-emulated sandbox, in: The Twelfth International Conference on Learning Representations, 2024, arXiv:2309.15817.

[12] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024, arXiv:2406.13352.

[13] A. S. Wijaya, J. Liu, H. Wang, E. Wu, SANA: What matters for QA agents over massive data lakes?, arXiv:2606.13904 (2026).

[14] R. Zhong, T. Yu, D. Klein, Semantic evaluation for text-to-SQL with distilled test suites, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 396–411. doi:10.18653/v1/2020.emnlp-main.29.

[15] F. T. Liu, K. M. Ting, Z.-H. Zhou, Isolation forest, in: Proceedings of the 8th IEEE International Conference on Data Mining, 2008, pp. 413–422. doi:10.1109/ICDM.2008.17.

[16] M. M. Breunig, H.-P. Kriegel, R. T. Ng, J. Sander, LOF: Identifying density-based local outliers, in: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 2000, pp. 93–104. doi:10.1145/342009.335388.

[17] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, R. C. Williamson, Estimating the support of a high-dimensional distribution, Neural Computation 13 (7) (2001) 1443–1471. doi:10.1162/089976601750264965.

[18] H. Kim, J. Taeyang, S. Choi, S. Choi, H. Cho, FLEX: Expert-level false-less execution metric for text-to-SQL benchmark, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for

Computational Linguistics: Human Language Technologies, 2025, pp. 4448–4475. doi:10.18653/v1/2025.naacl-long.228.

[19] Y. Wang, J. Zhang, T. Cai, Z. Liu, Q. Sun, Z. Sun, Z. Wu, M. Zhang, Y. Zhu, From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents, arXiv:2606.04990 (2026).

[20] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, G. Liu, R-Judge: Benchmarking safety risk awareness for LLM agents, arXiv:2401.10019 (2024).

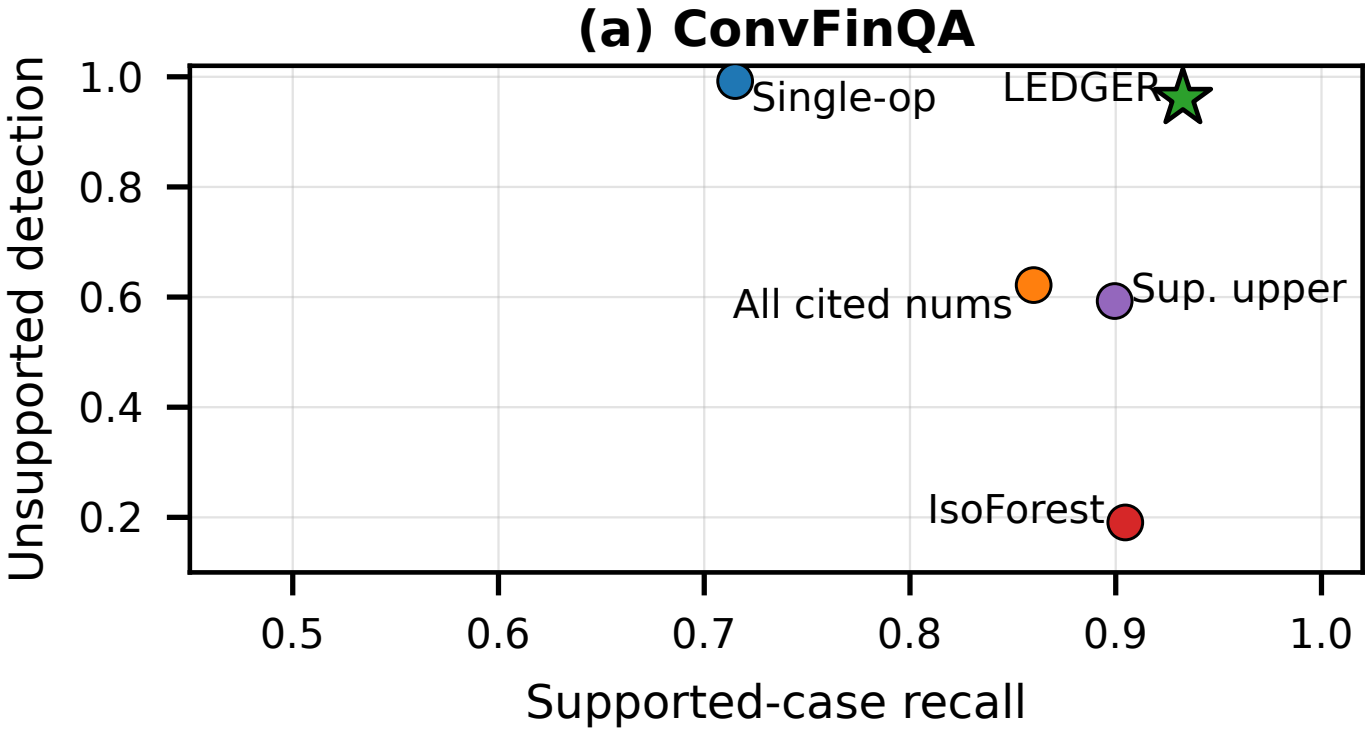
[21] A. Kamath, R. Jia, P. Liang, Selective question answering under domain shift, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 5684–5696. doi:10.18653/v1/2020.acl-main.503.

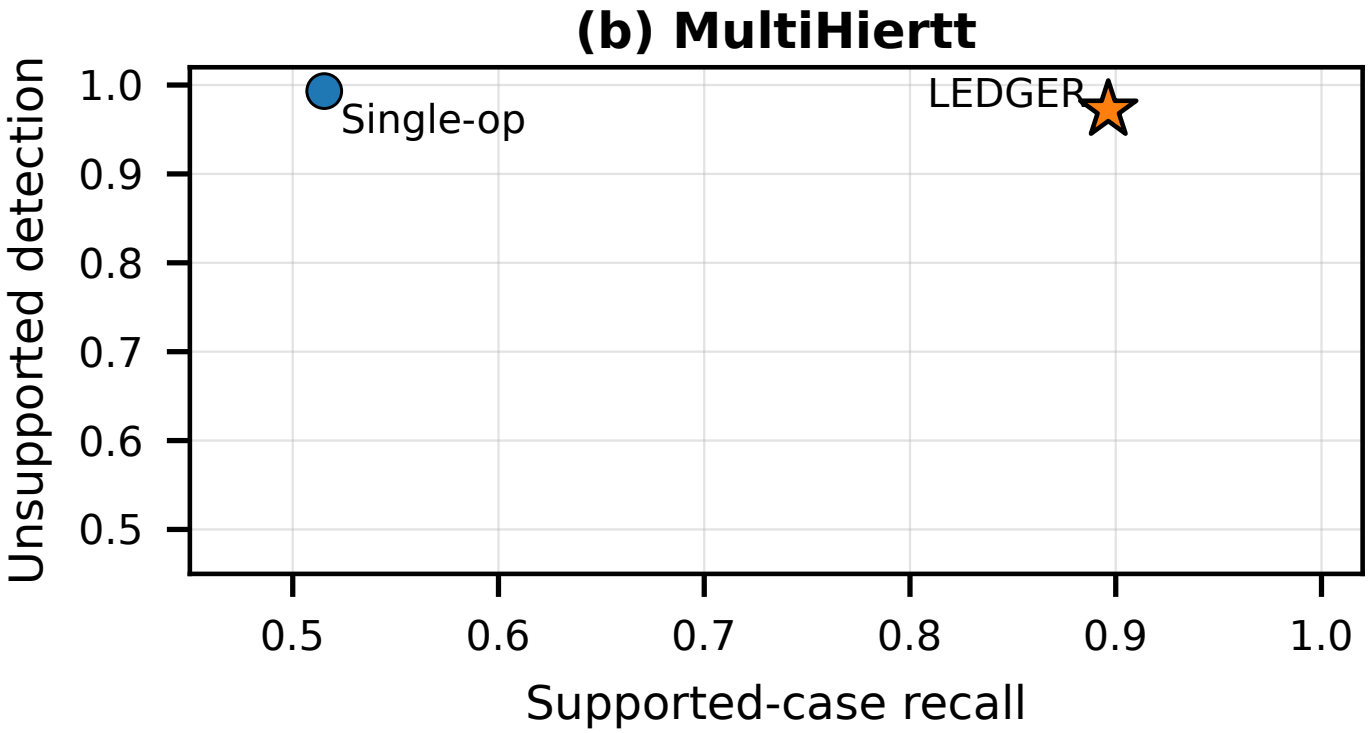
[22] Y. Nian, A. Yuan, H. Zhang, J. Li, Y. Zhao, Auditable agents, arXiv:2604.05485 (2026). doi:10.48550/arXiv.2604.05485.

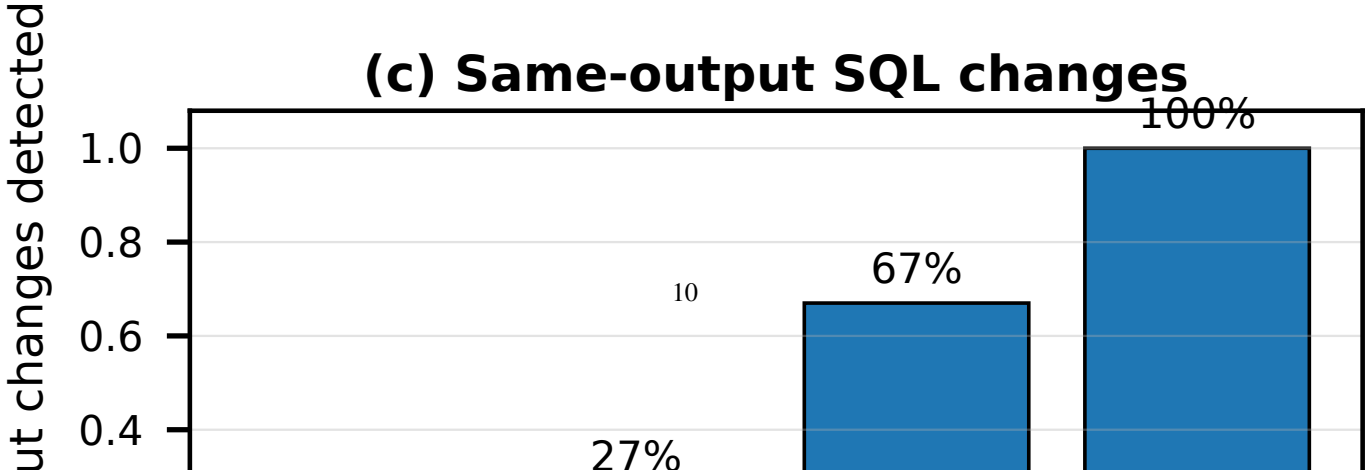
[23] V. Siu, J. He, K. Montgomery, Z. Wang, N. Gong, C. Wang, D. Song, A framework for formalizing LLM agent security, arXiv:2603.19469 (2026). doi:10.48550/arXiv.2603.19469.

[24] W. Qu, M. Xu, P. Wang, S. Zhai, J. Zhang, D. Song, Securing LLM agents need intent-to-execution integrity, arXiv:2605.16976 (2026). doi:10.48550/arXiv.2605.16976.

[25] W. Hua, X. Yang, M. Jin, Z. Li, W. Cheng, R. Tang, Y. Zhang, TrustAgent: Towards safe and trustworthy LLM-based agents, in: Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 10000–10016. doi:10.18653/v1/2024.findings-emnlp.585.


(a) ConvFinQA
Unsupported detection
1.0
0.8
0.6
0.4
0.2
Single-op
LEDGER
All cited nums
Sup. upper
IsoForest
0.5
0.6
0.7
0.8
0.9
1.0
Supported-case recall


(b) MultiHiertt
Unsupported detection
1.0
0.9
0.8
0.7
0.6
0.5
Single-op
LEDGER
0.5
0.6
0.7
0.8
0.9
1.0
Supported-case recall


(c) Same-output SQL changes
changes detected
1.0
0.8
0.6
0.4
67%
100%
27%
10

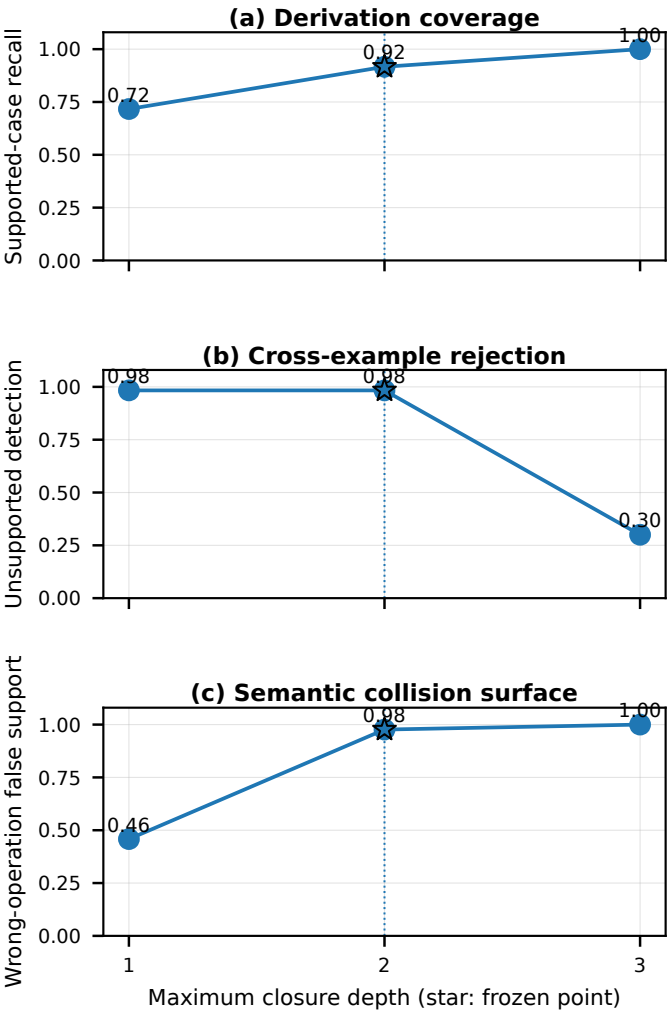


Figure 3: Closure depth trades derivation coverage for a larger collision surface; the star marks the frozen depth-2 setting.

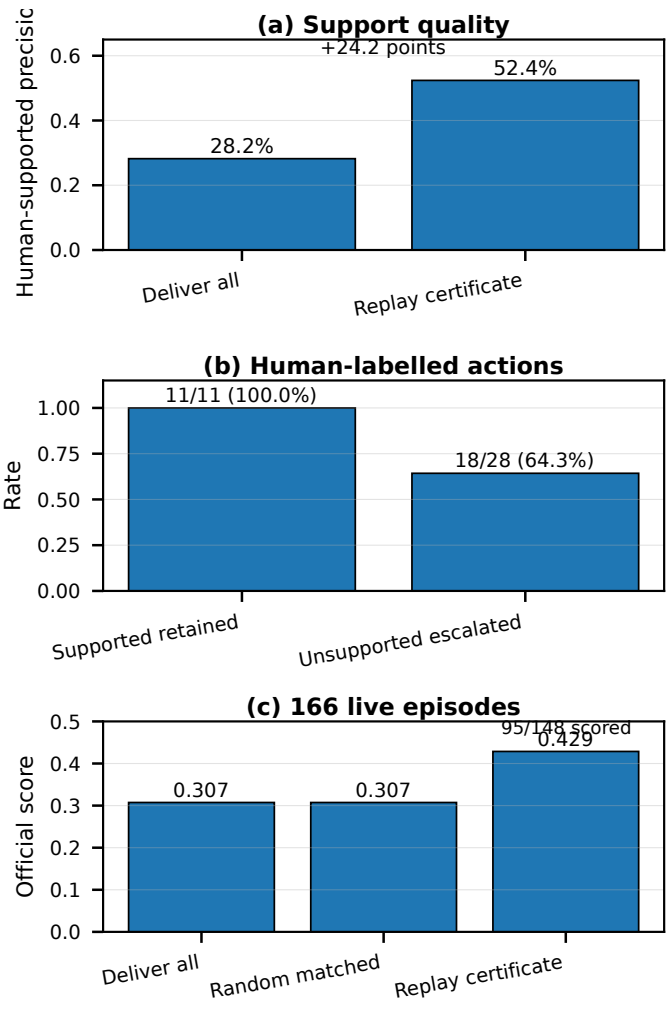


Figure 4: Replay-based delivery raises support precision, escalates unsupported cases, and improves live-episode utility at partial coverage.